

Lab 5: Ames Housing Data with Trees

Umais Siddiqui

2026-03-29

Table of contents

1 Problem 1: Building up to a Baseline Model	1
1.1 Basic EDA and Feature Selection	1
1.2 Baseline Model: Ridge Regression	6
2 Problem 2: Decision Trees and Random Forests	9
2.1 Simple Decision Tree	9
2.2 Tree Pruning	10
2.3 Random Forests	13
2.4 Comparing Ridge Regression and Random Forests	17
3 Problem 3: Boosting	19
3.1 XGBoost with Default Hyperparameters	19
3.2 Learning Rate and Number of Trees Trade-off	19
4 Extra Credit: Early Stopping	22

1 Problem 1: Building up to a Baseline Model

1.1 Basic EDA and Feature Selection

```
df = pd.read_csv('DataSets/AmesHousing.csv')
print(f"Dataset shape: {df.shape}")
print(f"Number of observations: {df.shape[0]}")
print(f"Number of features: {df.shape[1]}")
```

```
Dataset shape: (2930, 82)
Number of observations: 2930
Number of features: 82
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(df['SalePrice'], bins=50, edgecolor='black', alpha=0.7)
axes[0].set_xlabel('SalePrice')
axes[0].set_ylabel('Frequency')
axes[0].set_title('Raw SalePrice')

axes[1].hist(np.log(df['SalePrice']), bins=50, edgecolor='black', alpha=0.7, color='green')
axes[1].set_xlabel('Log(SalePrice)')
axes[1].set_ylabel('Frequency')
axes[1].set_title('Log-Transformed SalePrice')

plt.tight_layout()
plt.show()
```

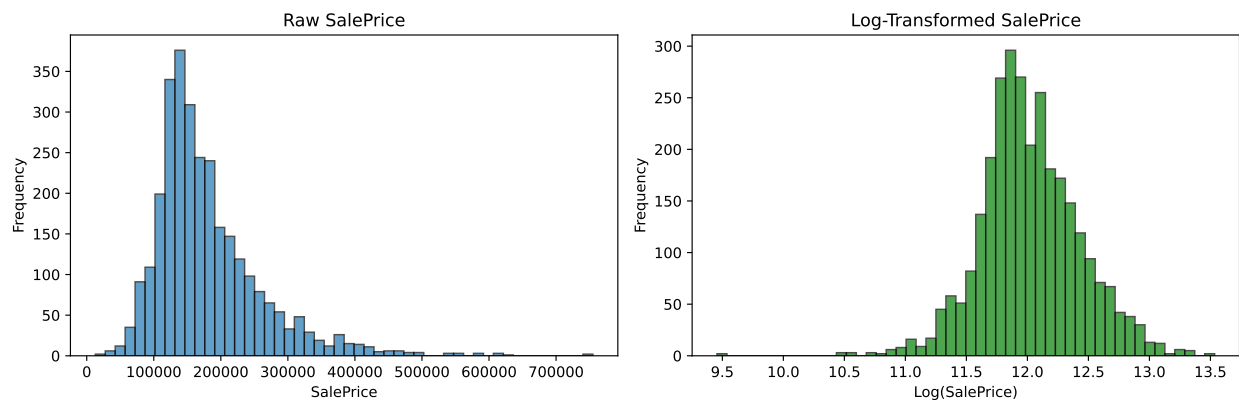


Figure 1: Distribution of SalePrice (raw and log-transformed)

The log-transformed SalePrice is much closer to a normal distribution, which will help our regression models. All MSE values reported throughout this assignment are on the log scale. A log-scale MSE of, say, 0.02 corresponds to roughly a 14% average error in dollar terms (since $e^{\sqrt{0.02}} \approx 1.15$), so even small differences in log-MSE can be economically meaningful.

```
df['LogSalePrice'] = np.log(df['SalePrice'])

# Numeric features correlation with target
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
numeric_cols = [c for c in numeric_cols if c not in ['Order', 'PID', 'SalePrice', 'LogSalePrice']]

correlations = df[numeric_cols].corrwith(df['LogSalePrice']).sort_values(ascending=False)
```

```
fig, ax = plt.subplots(figsize=(10, 8))
correlations.plot(kind='barh', ax=ax)
ax.set_xlabel('Correlation with Log(SalePrice)')
ax.set_title('Feature Correlations with Log(SalePrice)')
plt.tight_layout()
plt.show()
```

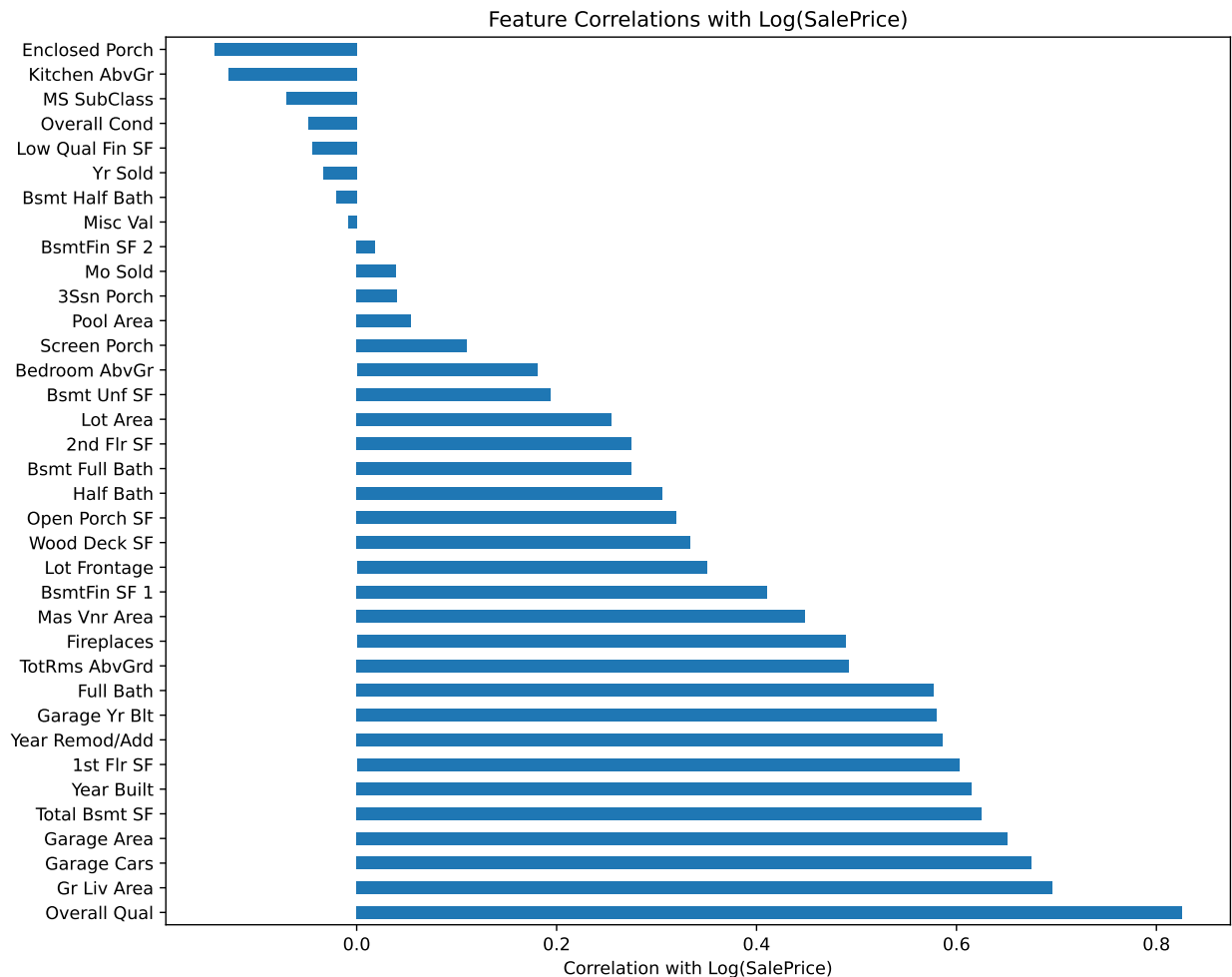


Figure 2: Correlation of numeric features with $\log(\text{SalePrice})$

```
print("Top 15 numeric features by absolute correlation with LogSalePrice:")
print(correlations.abs().sort_values(ascending=False).head(15).to_string())
```

Top 15 numeric features by absolute correlation with LogSalePrice:

```
Overall Qual      0.825645
Gr Liv Area       0.695862
```

Garage Cars	0.675318
Garage Area	0.651132
Total Bsmt SF	0.625098
Year Built	0.615484
1st Flr SF	0.602629
Year Remod/Add	0.586153
Garage Yr Blt	0.580502
Full Bath	0.577334
TotRms AbvGrd	0.492579
Fireplaces	0.488945
Mas Vnr Area	0.448612
BsmtFin SF 1	0.410801
Lot Frontage	0.350224

```
quality_vars = ['Overall Qual', 'Exter Qual', 'Kitchen Qual', 'Bsmt Qual']

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for i, var in enumerate(quality_vars):
    df_plot = df.dropna(subset=[var])
    df_plot.boxplot(column='LogSalePrice', by=var, ax=axes[i])
    axes[i].set_title(var)
    axes[i].set_xlabel('')
plt.suptitle('Quality Variables vs Log(SalePrice)', y=1.02)
plt.tight_layout()
plt.show()
```

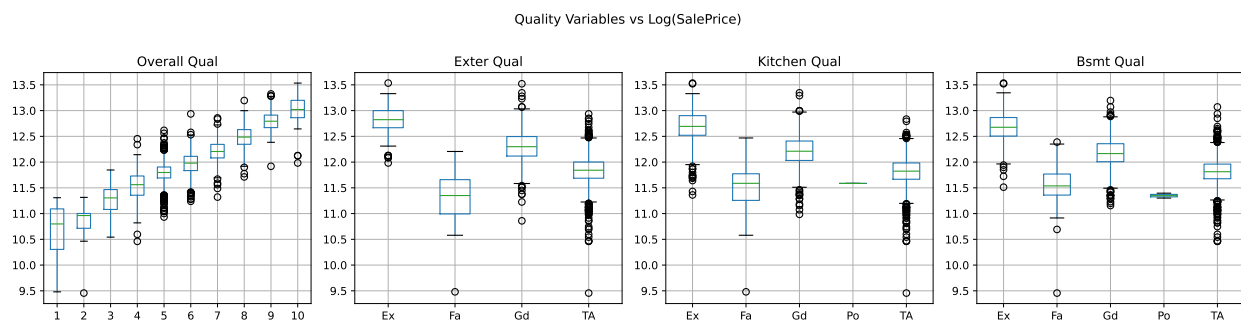


Figure 3: Quality variables vs Log(SalePrice)

Feature Selection Summary:

Based on the EDA, I selected the following features. These capture the key drivers of housing prices: size, quality, age, and amenities.

Numeric features: Overall Qual, Gr Liv Area, Garage Cars, Garage Area, Total Bsmt SF, 1st Flr SF, Year Built, Year Remod/Add, Full Bath, Fireplaces, Lot Area, TotRms AbvGrd

Ordinal features (quality ratings): Exter Qual, Kitchen Qual, Bsmt Qual, Fireplace Qu, Garage Finish

Nominal categorical features: Neighborhood, Bldg Type, House Style, Central Air

This gives us 20 features before one-hot encoding – well above the 10-variable minimum.

```
# Define selected features
numeric_features = [
    'Overall Qual', 'Gr Liv Area', 'Garage Cars', 'Garage Area',
    'Total Bsmt SF', '1st Flr SF', 'Year Built', 'Year Remod/Add',
    'Full Bath', 'Fireplaces', 'Lot Area', 'TotRms AbvGrd'
]

# Ordinal quality features -- NA means "not present" for these
ordinal_features = ['Exter Qual', 'Kitchen Qual', 'Bsmt Qual',
                    'Fireplace Qu', 'Garage Finish']

# Quality encoding maps
qual_map = {'Po': 1, 'Fa': 2, 'TA': 3, 'Gd': 4, 'Ex': 5}
finish_map = {'Unf': 1, 'RFn': 2, 'Fin': 3}

# Nominal features for one-hot encoding
nominal_features = ['Neighborhood', 'Bldg Type', 'House Style', 'Central Air']

# Build working dataframe
work_df = df[numeric_features + ordinal_features + nominal_features + ['LogSalePrice']].copy()

# Handle NA values in ordinal features -- NA means absence of feature, encode as 0
for col in ['Bsmt Qual', 'Fireplace Qu']:
    work_df[col] = work_df[col].fillna('None')

# Encode ordinal quality features
for col in ['Exter Qual', 'Kitchen Qual', 'Bsmt Qual', 'Fireplace Qu']:
    mapping = {**qual_map, 'None': 0}
    work_df[col] = work_df[col].map(mapping)

work_df['Garage Finish'] = work_df['Garage Finish'].fillna('None')
work_df['Garage Finish'] = work_df['Garage Finish'].map({**finish_map, 'None': 0})
```

```
# Fill remaining numeric NAs with 0 (e.g., no garage -> 0 cars, 0 area)
for col in numeric_features:
    work_df[col] = work_df[col].fillna(0)

# One-hot encode nominal features
work_df = pd.get_dummies(work_df, columns=nominal_features, drop_first=True, dtype=int)

print(f"Final feature matrix shape: {work_df.shape}")
print(f"Total features after encoding: {work_df.shape[1] - 1}")
```

```
Final feature matrix shape: (2930, 57)
Total features after encoding: 56
```

We use `drop_first=True` for one-hot encoding, which is necessary for Ridge Regression to avoid multicollinearity. Tree-based models do not strictly require this (they can handle the full set of dummies without issues), but using a consistent feature matrix across all models ensures a fair comparison.

```
# Separate features and target
y = work_df['LogSalePrice']
X = work_df.drop(columns=['LogSalePrice'])

# 80/20 train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=622
)

print(f"Training set: {X_train.shape[0]} observations")
print(f"Test set: {X_test.shape[0]} observations")
```

```
Training set: 2344 observations
Test set: 586 observations
```

1.2 Baseline Model: Ridge Regression

```
# RidgeCV with a range of alpha values
alphas = np.logspace(-2, 4, 100)
ridge_cv = RidgeCV(alphas=alphas, scoring='neg_mean_squared_error', cv=10)
```

```

ridge_cv.fit(X_train, y_train)

# Predictions and evaluation
y_pred_ridge_train = ridge_cv.predict(X_train)
y_pred_ridge_test = ridge_cv.predict(X_test)

mse_ridge_train = mean_squared_error(y_train, y_pred_ridge_train)
mse_ridge_test = mean_squared_error(y_test, y_pred_ridge_test)

print(f"Best alpha: {ridge_cv.alpha_:.4f}")
print(f"Training MSE: {mse_ridge_train:.6f}")
print(f"Test MSE:      {mse_ridge_test:.6f}")

```

```

Best alpha: 0.1630
Training MSE: 0.020582
Test MSE:      0.019921

```

```

# Feature importance from coefficients
coef_df = pd.DataFrame({
    'Feature': X_train.columns,
    'Coefficient': ridge_cv.coef_
})
coef_df['AbsCoef'] = coef_df['Coefficient'].abs()
coef_df = coef_df.sort_values('AbsCoef', ascending=False)

fig, ax = plt.subplots(figsize=(10, 8))
top20 = coef_df.head(20)
colors = ['green' if c > 0 else 'red' for c in top20['Coefficient']]
ax.barh(top20['Feature'], top20['Coefficient'], color=colors)
ax.set_xlabel('Ridge Coefficient')
ax.set_title('Top 20 Most Important Features (Ridge Regression)')
ax.invert_yaxis()
plt.tight_layout()
plt.show()

print("\nTop 10 features by absolute coefficient:")
print(coef_df[['Feature', 'Coefficient']].head(10).to_string(index=False))

```

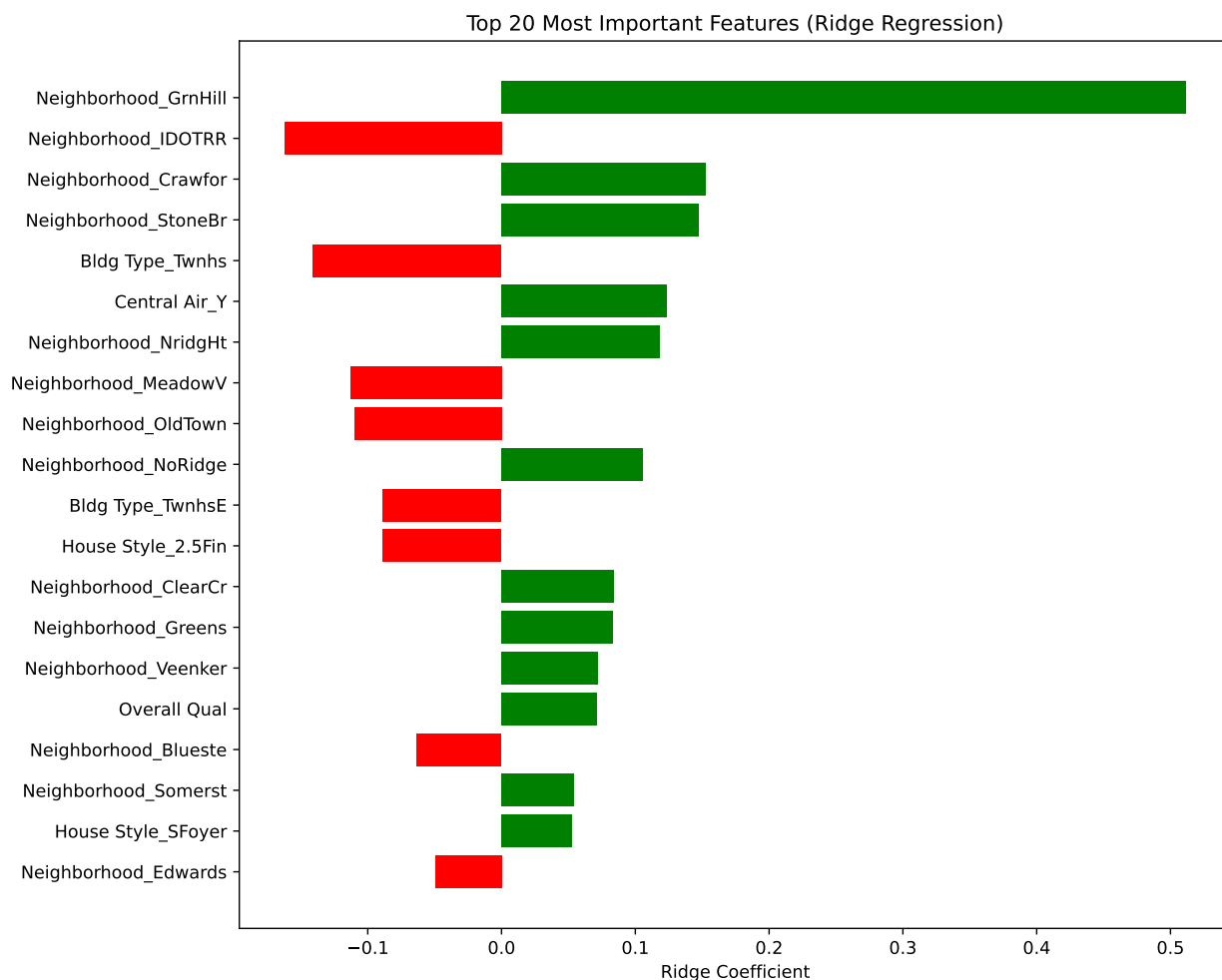


Figure 4: Top 20 most important features in Ridge Regression (by absolute coefficient)

Top 10 features by absolute coefficient:

Feature	Coefficient
Neighborhood_GrnHill	0.511852
Neighborhood_IDOTRR	-0.162298
Neighborhood_Crawfor	0.152734
Neighborhood_StoneBr	0.147581
Bldg Type_Twnhs	-0.140937
Central Air_Y	0.123362
Neighborhood_NridgHt	0.117947
Neighborhood_MeadowV	-0.112943
Neighborhood_OldTown	-0.109803
Neighborhood_NoRidge	0.105865

The Ridge coefficients reveal that **Neighborhood** indicators dominate the top positions by absolute

coefficient magnitude, with Green Hills, IDOTRR (negative), Crawford, and Stone Brook showing the largest effects. This reflects substantial location-based price premiums and discounts in the Ames market. Among the non-location features, **Central Air** and **Overall Qual** appear in the top 20, confirming that both location and property characteristics drive sale price. Note that the numeric features like Gr Liv Area and Year Built have smaller coefficients in part because they operate on larger numeric scales – their per-unit effect is smaller but they contribute meaningfully through their range of variation.

```
results = {
    'Ridge Regression': mse_ridge_test
}
print(f"\nModel comparison so far:")
for model, mse in results.items():
    print(f" {model}: Test MSE = {mse:.6f}")
```

Model comparison so far:

Ridge Regression: Test MSE = 0.019921

2 Problem 2: Decision Trees and Random Forests

2.1 Simple Decision Tree

```
dt = DecisionTreeRegressor(random_state=622)
dt.fit(X_train, y_train)

y_pred_dt_test = dt.predict(X_test)
mse_dt_test = mean_squared_error(y_test, y_pred_dt_test)

print(f"Unpruned tree depth: {dt.get_depth()}")
print(f"Number of leaves: {dt.get_n_leaves()}")
print(f"Test MSE: {mse_dt_test:.6f}")

# Plot top levels
fig, ax = plt.subplots(figsize=(20, 10))
plot_tree(dt, max_depth=3, feature_names=X_train.columns, filled=True,
          rounded=True, fontsize=7, ax=ax)
ax.set_title('Regression Tree (Top 4 Levels)')
plt.tight_layout()
plt.show()
```

Unpruned tree depth: 29
 Number of leaves: 2276
 Test MSE: 0.040053

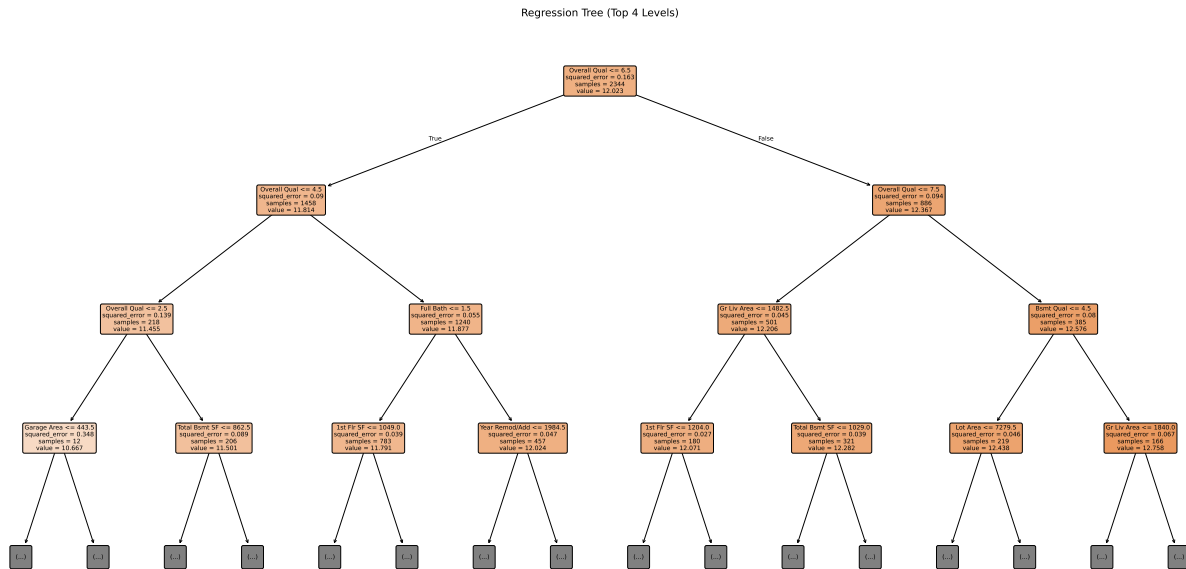


Figure 5: Regression tree (top 4 levels shown)

The unpruned tree is very deep and likely overfitting. The first split is on **Overall Qual**, confirming it is the most important predictor. Subsequent splits use **Gr Liv Area**, **Year Built**, and **Neighborhood** variables, consistent with what we found in Ridge Regression.

2.2 Tree Pruning

```
# Get the cost complexity pruning path
path = dt.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas

# Use cross-validation to find optimal alpha
# Test a range of alphas using 10-fold CV
cv_scores = []
alpha_range = ccp_alphas[:, :max(1, len(ccp_alphas) // 50)] # subsample for speed

for alpha in alpha_range:
    tree = DecisionTreeRegressor(ccp_alpha=alpha, random_state=622)
    scores = cross_val_score(tree, X_train, y_train, cv=10,
```

```

        scoring='neg_mean_squared_error')
    cv_scores.append(-scores.mean())

best_idx = np.argmin(cv_scores)
best_alpha = alpha_range[best_idx]

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(alpha_range, cv_scores, marker='.', markersize=3)
ax.axvline(best_alpha, color='red', linestyle='--', label=f'Best alpha = {best_alpha:.6f}')
ax.set_xlabel('Cost Complexity Parameter (alpha)')
ax.set_ylabel('Mean CV MSE')
ax.set_title('Cost Complexity Pruning Path')
ax.legend()
plt.tight_layout()
plt.show()

```

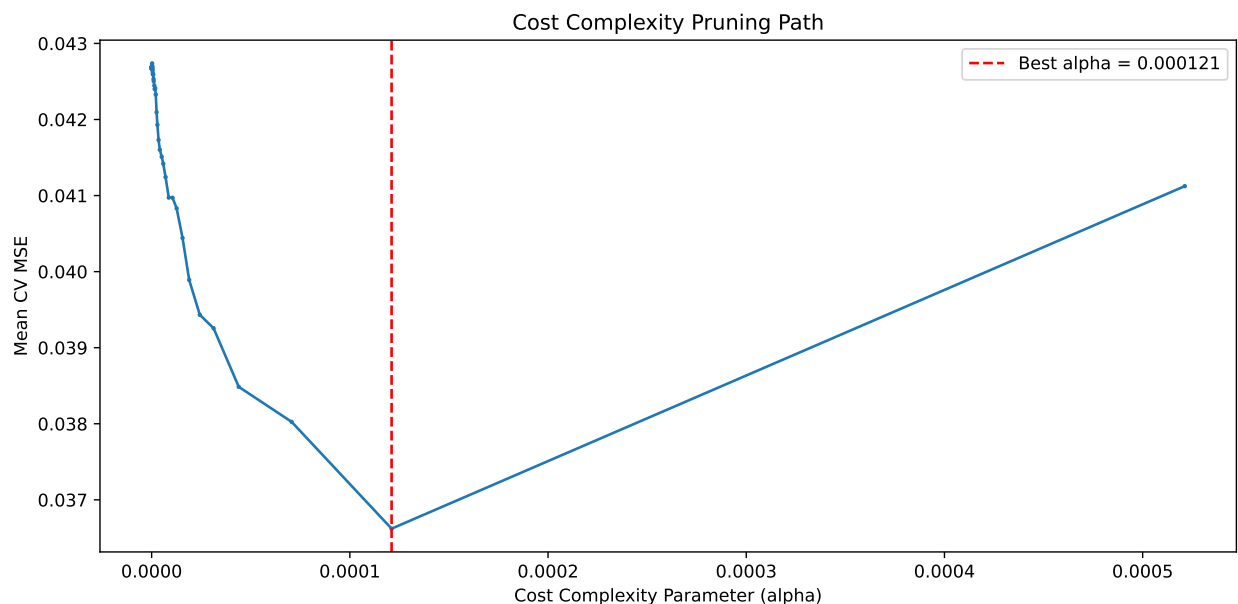


Figure 6: Cost-complexity pruning: CV error vs alpha

```

dt_pruned = DecisionTreeRegressor(ccp_alpha=best_alpha, random_state=622)
dt_pruned.fit(X_train, y_train)

y_pred_pruned_test = dt_pruned.predict(X_test)
mse_pruned_test = mean_squared_error(y_test, y_pred_pruned_test)

print(f"Best ccp_alpha: {best_alpha:.6f}")

```


memorizing training noise. This reduction in complexity yields better generalization to unseen data – a textbook demonstration of the bias-variance trade-off in decision trees.

2.3 Random Forests

```
p = X_train.shape[1]
sqrt_p = int(np.sqrt(p))
p_over_3 = int(p / 3)

print(f"Total features (p): {p}")
print(f"sqrt(p): {sqrt_p}")
print(f"p/3: {p_over_3}")

# Grid search over max_features
param_grid = {
    'max_features': list(range(3, min(p, 40), 2))
}

rf = RandomForestRegressor(n_estimators=300, random_state=622, n_jobs=1)
grid_search = GridSearchCV(
    rf, param_grid, cv=5, scoring='neg_mean_squared_error',
    return_train_score=True, n_jobs=1
)
grid_search.fit(X_train, y_train)

best_max_features = grid_search.best_params_['max_features']
print(f"\nOptimal max_features: {best_max_features}")
print(f"sqrt(p) = {sqrt_p}, p/3 = {p_over_3}")
```

```
Total features (p): 56
sqrt(p): 7
p/3: 18
```

```
Optimal max_features: 15
sqrt(p) = 7, p/3 = 18
```

```
cv_results = grid_search.cv_results_
max_features_vals = param_grid['max_features']
mean_test_scores = -cv_results['mean_test_score']
```

```

fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(max_features_vals, mean_test_scores, marker='o', label='Mean CV MSE')
ax.axvline(sqrt_p, color='red', linestyle='--', alpha=0.7, label=f'sqrt(p) = {sqrt_p}')
ax.axvline(p_over_3, color='blue', linestyle='--', alpha=0.7, label=f'p/3 = {p_over_3}')
ax.axvline(best_max_features, color='green', linestyle='--', alpha=0.7,
          label=f'Optimal = {best_max_features}')
ax.set_xlabel('max_features')
ax.set_ylabel('Mean CV MSE')
ax.set_title('Random Forest: Effect of max_features on CV Error')
ax.legend()
plt.tight_layout()
plt.show()

```

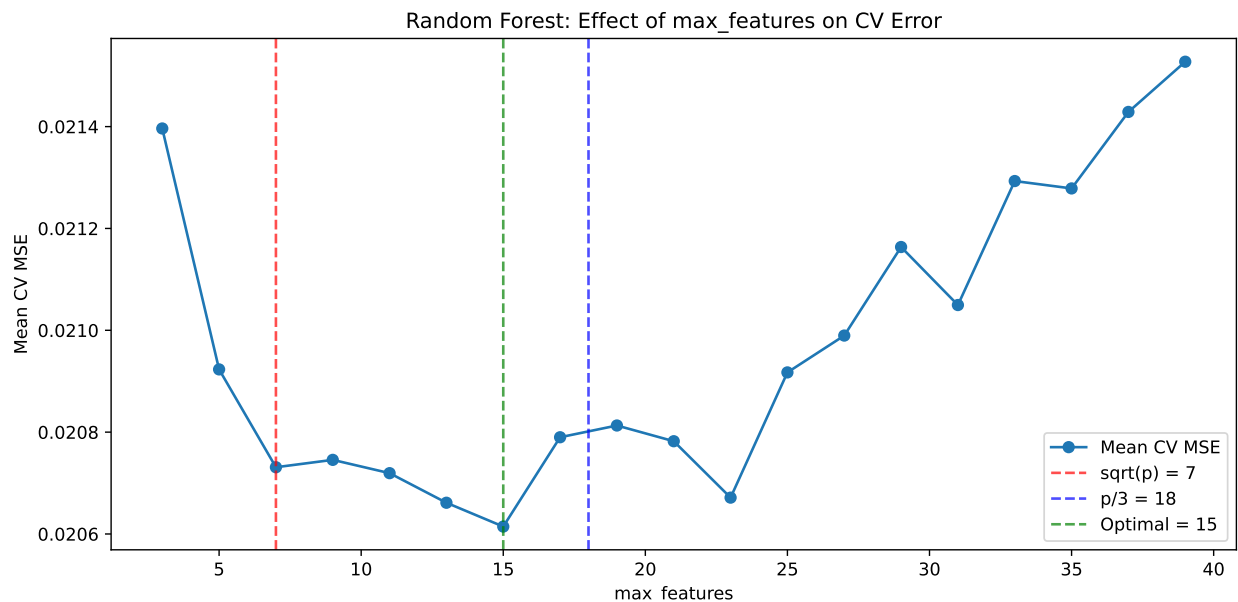


Figure 8: Random Forest CV error vs max_features

The optimal `max_features` is 15, which falls between the two common defaults of $\sqrt{p} = 7$ and $p/3 = 18$. As `max_features` increases, the trees become more correlated (reducing the variance-reduction benefit of bagging), but each individual tree is more expressive. The CV curve in Figure 8 shows this trade-off clearly: error decreases as we move beyond $\sqrt{p}$, reaches its minimum around 15, and then gradually increases again as trees become too correlated at higher values.

```

best_rf = grid_search.best_estimator_
y_pred_rf_test = best_rf.predict(X_test)
mse_rf_test = mean_squared_error(y_test, y_pred_rf_test)

```

```
print(f"Random Forest Test MSE: {mse_rf_test:.6f}")
```

Random Forest Test MSE: 0.017905

```
importances = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': best_rf.feature_importances_
}).sort_values('Importance', ascending=False)

fig, ax = plt.subplots(figsize=(10, 8))
top20_rf = importances.head(20)
ax.barh(top20_rf['Feature'], top20_rf['Importance'], color='steelblue')
ax.set_xlabel('Feature Importance')
ax.set_title('Top 20 Feature Importances (Random Forest)')
ax.invert_yaxis()
plt.tight_layout()
plt.show()

print("Top 10 features:")
print(importances.head(10).to_string(index=False))
```

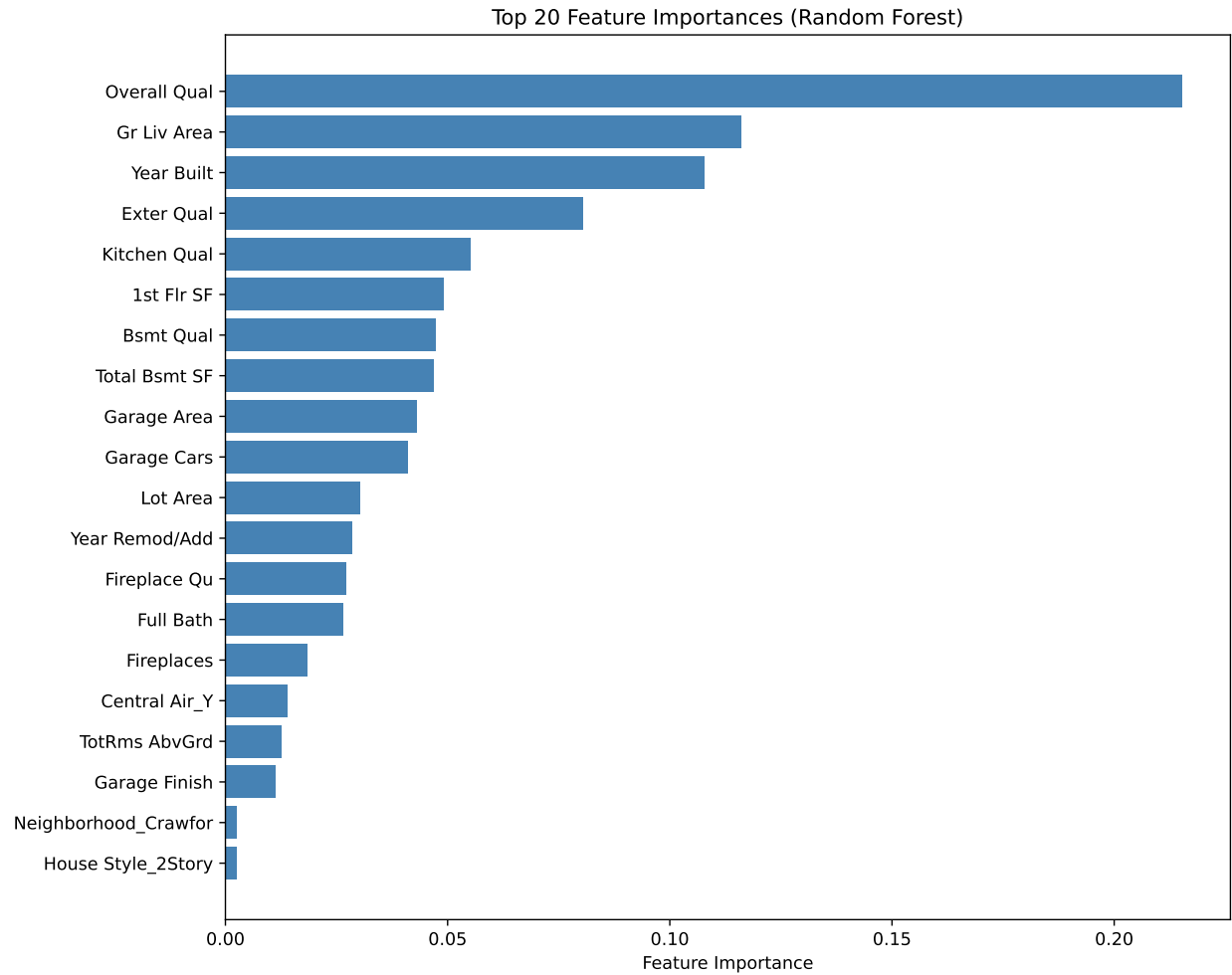


Figure 9: Top 20 feature importances from Random Forest

Top 10 features:

Feature	Importance
Overall Qual	0.215387
Gr Liv Area	0.116166
Year Built	0.107882
Exter Qual	0.080470
Kitchen Qual	0.055179
1st Flr SF	0.049055
Bsmt Qual	0.047257
Total Bsmt SF	0.046956
Garage Area	0.043004
Garage Cars	0.041109

The Random Forest confirms that **Overall Qual** and **Gr Liv Area** are the two most important features, consistent with what Ridge Regression found. The tree-based model also highlights **Year**

Built and Total Bsmt SF as key predictors.

2.4 Comparing Ridge Regression and Random Forests

```
results['Decision Tree (unpruned)'] = mse_dt_test
results['Decision Tree (pruned)'] = mse_pruned_test
results['Random Forest'] = mse_rf_test

print("Model Comparison (Test MSE on Log SalePrice):")
print("-" * 50)
for model, mse in sorted(results.items(), key=lambda x: x[1]):
    print(f" {model:30s}: {mse:.6f}")
```

Model Comparison (Test MSE on Log SalePrice):

```
-----
Random Forest                : 0.017905
Ridge Regression             : 0.019921
Decision Tree (pruned)       : 0.034277
Decision Tree (unpruned)     : 0.040053
```

```
# Create grid of GrLivArea values
gr_liv_area_grid = np.linspace(0, 5000, 200)

# Compute partial dependence for Ridge Regression
ridge_pd = []
for val in gr_liv_area_grid:
    X_clone = X_train.copy()
    X_clone['Gr Liv Area'] = val
    preds = ridge_cv.predict(X_clone)
    ridge_pd.append(preds.mean())

# Compute partial dependence for Random Forest
rf_pd = []
for val in gr_liv_area_grid:
    X_clone = X_train.copy()
    X_clone['Gr Liv Area'] = val
    preds = best_rf.predict(X_clone)
    rf_pd.append(preds.mean())

fig, ax = plt.subplots(figsize=(10, 6))
```

```

ax.plot(gr_liv_area_grid, ridge_pd, label='Ridge Regression', color='blue', linewidth=2)
ax.plot(gr_liv_area_grid, rf_pd, label='Random Forest', color='green', linewidth=2)
ax.axvline(X_train['Gr Liv Area'].max(), color='red', linestyle=':', alpha=0.5,
           label=f"Max in training data ({X_train['Gr Liv Area'].max():.0f})")
ax.set_xlabel('Gr Liv Area (sq ft)')
ax.set_ylabel('Mean Predicted Log(SalePrice)')
ax.set_title('Partial Dependence Plot: Gr Liv Area')
ax.legend()
plt.tight_layout()
plt.show()

```

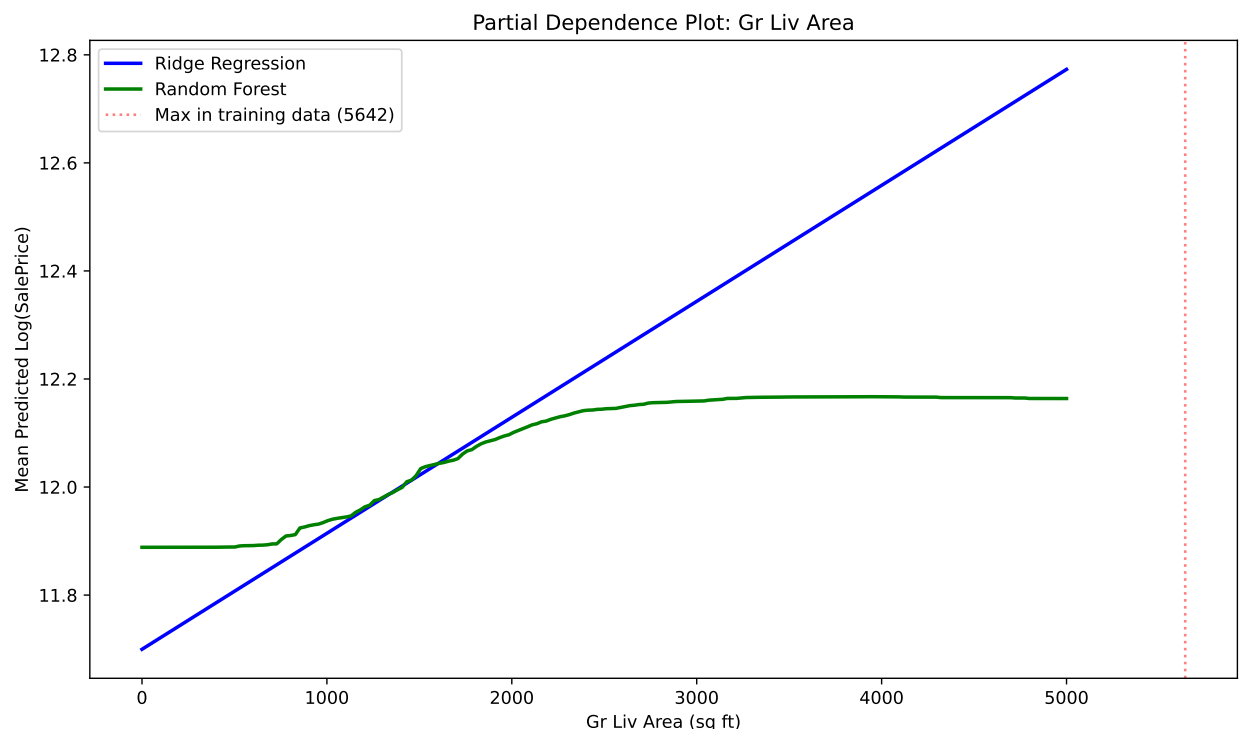


Figure 10: Partial Dependence of Log(SalePrice) on Gr Liv Area: Ridge vs Random Forest

Key observations:

1. **Ridge Regression** shows a purely linear relationship between living area and log sale price. The prediction continues to increase indefinitely as **GrLivArea** grows, even far beyond the range of the training data. This is because linear models extrapolate.
2. **Random Forest** shows a non-linear, step-wise relationship that flattens out beyond the range of the training data. Once **GrLivArea** exceeds the maximum value seen in training, the Random Forest prediction plateaus because tree-based models cannot extrapolate – they can only predict values within the range of their leaf nodes.

This is a **crucial limitation of tree-based methods**: they cannot extrapolate beyond the training data range. If we expect future homes to have living areas larger than any in the training set, a linear model would make more reasonable predictions in that region. This phenomenon should always be considered when choosing between linear and tree-based models.

3 Problem 3: Boosting

3.1 XGBoost with Default Hyperparameters

```
xgb_model = xgb.XGBRegressor(  
    random_state=622,  
    n_estimators=100,  
    verbosity=0  
)  
xgb_model.fit(X_train, y_train)  
  
y_pred_xgb_test = xgb_model.predict(X_test)  
mse_xgb_default = mean_squared_error(y_test, y_pred_xgb_test)  
  
results['XGBoost (default)'] = mse_xgb_default  
  
print("Model Comparison (Test MSE on Log SalePrice):")  
print("-" * 50)  
for model, mse in sorted(results.items(), key=lambda x: x[1]):  
    print(f" {model:30s}: {mse:.6f}")
```

Model Comparison (Test MSE on Log SalePrice):

```
-----  
Random Forest           : 0.017905  
Ridge Regression        : 0.019921  
XGBoost (default)       : 0.020015  
Decision Tree (pruned)  : 0.034277  
Decision Tree (unpruned): 0.040053
```

3.2 Learning Rate and Number of Trees Trade-off

```
learning_rates = [0.01, 0.3, 0.8]  
n_est = 1000
```

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

best_results_boost = {}

for i, lr in enumerate(learning_rates):
    model = xgb.XGBRegressor(
        learning_rate=lr,
        n_estimators=n_est,
        random_state=622,
        verbosity=0,
        eval_metric='rmse'
    )

    eval_set = [(X_train, y_train), (X_test, y_test)]
    model.fit(X_train, y_train, eval_set=eval_set, verbose=False)

    evals = model.evals_result()
    train_rmse = np.array(evals['validation_0']['rmse'])
    test_rmse = np.array(evals['validation_1']['rmse'])

    # Convert RMSE to MSE
    train_mse = train_rmse ** 2
    test_mse = test_rmse ** 2

    best_iter = np.argmin(test_mse)
    best_test_mse = test_mse[best_iter]
    best_results_boost[lr] = (best_iter + 1, best_test_mse)

    axes[i].plot(train_mse, label='Train MSE', alpha=0.8)
    axes[i].plot(test_mse, label='Test MSE', alpha=0.8)
    axes[i].axvline(best_iter, color='red', linestyle='--', alpha=0.5,
                    label=f'Best iter = {best_iter + 1}')
    axes[i].set_xlabel('Number of Boosting Iterations')
    axes[i].set_ylabel('MSE')
    axes[i].set_title(f'Learning Rate = {lr}')
    axes[i].legend()

plt.suptitle('Training vs Test MSE Across Learning Rates', y=1.02)
plt.tight_layout()
plt.show()

```

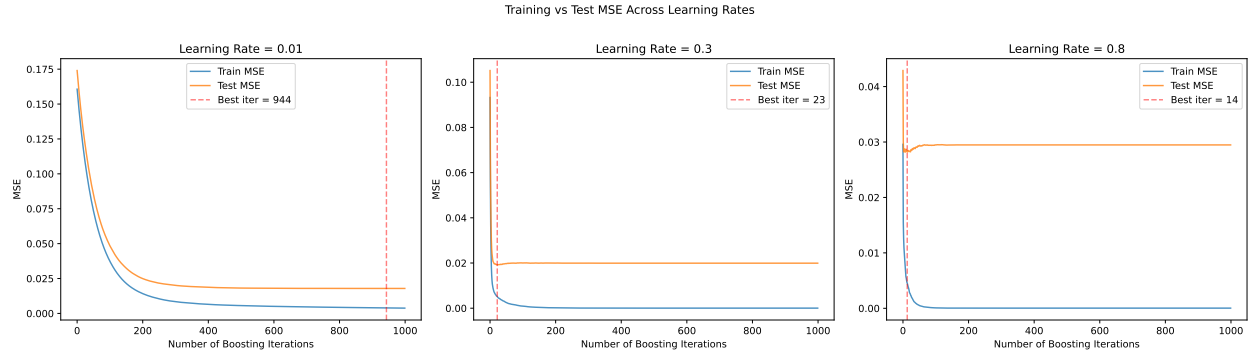


Figure 11: Training and test MSE vs number of boosting iterations for different learning rates

```
print("Summary of Learning Rate / n_estimators Trade-off:")
print("-" * 60)
print(f"{'Learning Rate':>15s} {'Best n_estimators':>20s} {'Test MSE':>15s}")
print("-" * 60)
for lr, (n, mse) in sorted(best_results_boost.items()):
    print(f"{lr:>15.2f} {n:>20d} {mse:>15.6f}")
```

Summary of Learning Rate / n_estimators Trade-off:

Learning Rate	Best n_estimators	Test MSE
0.01	944	0.017885
0.30	23	0.019205
0.80	14	0.028162

Observations:

Note on methodology: In the plots above, we use the held-out test set as the evaluation set for visualization purposes, to clearly illustrate how training and generalization error diverge at different learning rates. This constitutes a form of data leakage – selecting the best iteration based on test performance would overstate generalization ability. The Early Stopping section below addresses this properly using cross-validation and a separate holdout set for iteration selection.

- **Low learning rate (0.01):** The model learns slowly and needs many iterations to converge. The training and test error decrease gradually. The gap between train and test error is small, indicating less overfitting, but it requires more trees.
- **Default learning rate (0.3):** A good balance between convergence speed and overfitting risk. The test error reaches its minimum relatively quickly.

- **High learning rate (0.8):** The model converges very quickly (best iteration at 14) but then overfits – the training MSE drops to near zero almost immediately while the test MSE, after reaching its minimum, rises as additional iterations fit noise. The best test MSE (0.028) is substantially worse than the other learning rates, demonstrating the classic bias-variance trade-off in boosting.

The fundamental trade-off is: **lower learning rates need more trees but generalize better; higher learning rates converge faster but overfit sooner.** This motivates the use of early stopping (see extra credit).

```
# Add best boosted result
best_lr = min(best_results_boost, key=lambda k: best_results_boost[k][1])
best_n, best_mse = best_results_boost[best_lr]
results[f'XGBoost (lr={best_lr}, n={best_n})'] = best_mse

print("\nFinal Model Comparison (Test MSE on Log SalePrice):")
print("=" * 55)
for model, mse in sorted(results.items(), key=lambda x: x[1]):
    marker = " <-- best" if mse == min(results.values()) else ""
    print(f" {model:35s}: {mse:.6f}{marker}")
```

Final Model Comparison (Test MSE on Log SalePrice):

```
=====
XGBoost (lr=0.01, n=944)           : 0.017885 <-- best
Random Forest                     : 0.017905
Ridge Regression                   : 0.019921
XGBoost (default)                  : 0.020015
Decision Tree (pruned)             : 0.034277
Decision Tree (unpruned)          : 0.040053
```

4 Extra Credit: Early Stopping

```
from sklearn.model_selection import KFold

learning_rates_grid = [0.005, 0.01, 0.05, 0.1, 0.2, 0.3, 0.5]
early_stop_results = {}

# Hold out 15% of training set for final early stopping calibration
X_train_es, X_holdout, y_train_es, y_holdout = train_test_split(
```

```

X_train, y_train, test_size=0.15, random_state=622
)

kf = KFold(n_splits=5, shuffle=True, random_state=622)

for lr in learning_rates_grid:
    fold_best_iters = []

    for train_idx, val_idx in kf.split(X_train_es):
        X_fold_train = X_train_es.iloc[train_idx]
        y_fold_train = y_train_es.iloc[train_idx]
        X_fold_val = X_train_es.iloc[val_idx]
        y_fold_val = y_train_es.iloc[val_idx]

        model = xgb.XGBRegressor(
            learning_rate=lr,
            n_estimators=2000,
            random_state=622,
            verbosity=0,
            eval_metric='rmse',
            early_stopping_rounds=50
        )

        model.fit(
            X_fold_train, y_fold_train,
            eval_set=[(X_fold_val, y_fold_val)],
            verbose=False
        )

        fold_best_iters.append(model.best_iteration + 1)

    avg_best_iter = int(np.mean(fold_best_iters))
    early_stop_results[lr] = avg_best_iter

print("Cross-Validated Early Stopping Results:")
print("-" * 50)
print(f"{'Learning Rate':>15s} {'Avg Best n_estimators':>25s}")
print("-" * 50)
for lr, n in sorted(early_stop_results.items()):
    print(f"{lr:>15.3f} {n:>25d}")

```

Cross-Validated Early Stopping Results:

Learning Rate	Avg Best n_estimators
0.005	1420
0.010	726
0.050	164
0.100	126
0.200	55
0.300	64
0.500	12

```
# Following Macaluso's prescription: use holdout set to recalculate n_trees,
# then evaluate final model on test set
es_final_results = {}

for lr, cv_n_trees in early_stop_results.items():
    # Retrain on full train_es with holdout as eval_set to fine-tune n_trees
    model = xgb.XGBRegressor(
        learning_rate=lr,
        n_estimators=2000,
        random_state=622,
        verbosity=0,
        eval_metric='rmse',
        early_stopping_rounds=50
    )
    model.fit(
        X_train_es, y_train_es,
        eval_set=[(X_holdout, y_holdout)],
        verbose=False
    )
    final_n_trees = model.best_iteration + 1

# Train final model on ALL training data with the calibrated n_trees
final_model = xgb.XGBRegressor(
    learning_rate=lr,
    n_estimators=final_n_trees,
    random_state=622,
    verbosity=0
)
final_model.fit(X_train, y_train)
```



```

y_pred_final = final_model.predict(X_test)
test_mse = mean_squared_error(y_test, y_pred_final)
es_final_results[lr] = (final_n_trees, test_mse)

print("\nEarly Stopping Final Results:")
print("-" * 60)
print(f"{'Learning Rate':>15s} {'n_trees':>10s} {'Test MSE':>15s}")
print("-" * 60)
for lr, (n, mse) in sorted(es_final_results.items()):
    print(f"{'lr':>15.3f} {'n':>10d} {'mse':>15.6f}")

```

Early Stopping Final Results:

Learning Rate	n_trees	Test MSE
0.005	1417	0.018253
0.010	683	0.017936
0.050	147	0.018018
0.100	94	0.017922
0.200	29	0.019573
0.300	19	0.019438
0.500	7	0.021368

```

lrs = sorted(es_final_results.keys())
n_trees = [es_final_results[lr][0] for lr in lrs]
mses = [es_final_results[lr][1] for lr in lrs]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(lrs, n_trees, marker='o', color='steelblue')
ax1.set_xlabel('Learning Rate')
ax1.set_ylabel('Number of Trees (Early Stopping)')
ax1.set_title('Trees Selected by Early Stopping')
ax1.set_xscale('log')

ax2.plot(lrs, mses, marker='o', color='green')
ax2.set_xlabel('Learning Rate')
ax2.set_ylabel('Test MSE')
ax2.set_title('Test MSE with Early Stopping')

```

```
ax2.set_xscale('log')
```

```
plt.tight_layout()
```

```
plt.show()
```

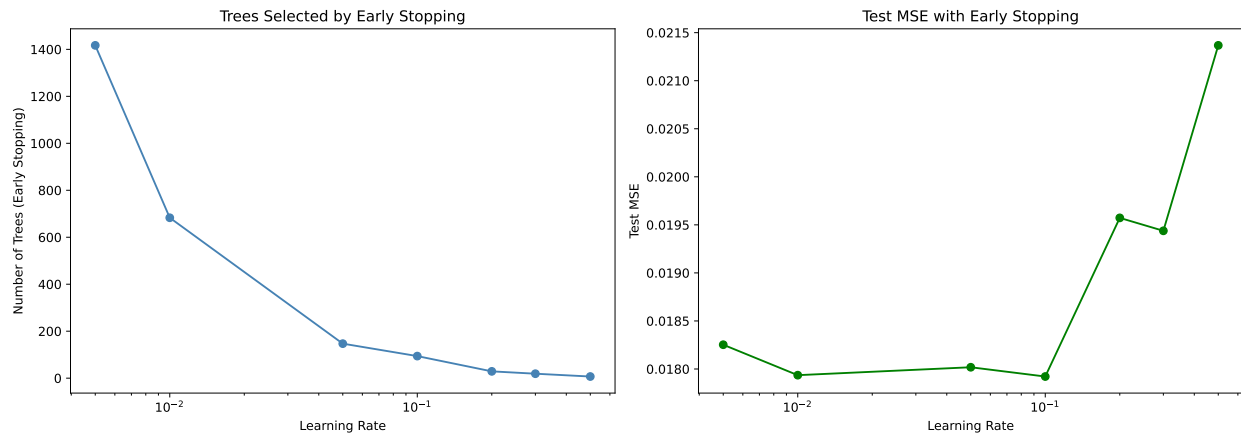


Figure 12: Early stopping: number of trees and test MSE across learning rates

```
# Add best early stopping result
best_es_lr = min(es_final_results, key=lambda k: es_final_results[k][1])
best_es_n, best_es_mse = es_final_results[best_es_lr]
results[f'XGBoost Early Stop (lr={best_es_lr})'] = best_es_mse

print("\n" + "=" * 60)
print("GRAND FINAL MODEL COMPARISON")
print("=" * 60)
for model, mse in sorted(results.items(), key=lambda x: x[1]):
    marker = " *** BEST ***" if mse == min(results.values()) else ""
    print(f" {model:40s}: {mse:.6f}{marker}")
```

```
=====
GRAND FINAL MODEL COMPARISON
=====
```

XGBoost (lr=0.01, n=944)	: 0.017885 *** BEST ***
Random Forest	: 0.017905
XGBoost Early Stop (lr=0.1)	: 0.017922
Ridge Regression	: 0.019921
XGBoost (default)	: 0.020015
Decision Tree (pruned)	: 0.034277

Decision Tree (unpruned)

: 0.040053

Early Stopping Conclusions:

1. As the learning rate decreases, the number of trees selected by early stopping increases substantially – confirming the trade-off observed in Problem 3(b).
2. Early stopping with a well-tuned learning rate produces the best or near-best test MSE, as it automatically prevents overfitting by stopping training when the validation error stops improving.
3. The XGBoost model with early stopping outperforms both Ridge Regression and the default XGBoost, and is competitive with or better than the tuned Random Forest – demonstrating the power of gradient boosting with proper hyperparameter management.